

APUNTESWEB · INGENIERÍA INFORMÁTICA

The logo consists of the word "SISIN" in a bold, black, serif font, centered within a solid yellow rectangular background.

Cuarto curso · 1.º cuatrimestre · 1 temas

SEPTIEMBRE DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Temple Simulado

1.1 Agentes Inteligentes

El tema arranca situando dónde encaja la búsqueda dentro de la Inteligencia Artificial (IA), siguiendo el libro de **Russell y Norvig**. Después se centra en las estrategias de búsqueda: **búsqueda local y heurística** y **metaheurísticas basadas en trayectorias**, en concreto el **enfriamiento simulado** (la búsqueda tabú aparece en el índice pero no se desarrolla en estas diapositivas).

1.1.1 Cuatro enfoques de la IA

Las definiciones de IA se pueden clasificar según dos preguntas: si se busca que la máquina **piense** o que **actúe**, y si el modelo es el **ser humano** o la **racionalidad** ideal. De ahí salen cuatro enfoques. **Pensar como humanos** corresponde al **modelado cognitivo**; **pensar racionalmente**, a la **lógica y las leyes del pensamiento**; **actuar como humanos**, al **test de Turing**, que exige capacidades como procesamiento del lenguaje natural (NLP), representación del conocimiento y razonamiento (KRR) y aprendizaje automático (ML); y **actuar racionalmente**, a los **agentes racionales**, que es el enfoque que sigue la asignatura.

Modelado
cognitivo



Lógica, leyes del
pensamiento



Thinking Humanly “The exciting new effort to make computers think . . . <i>machines with minds</i> , in the full and literal sense.” (Haugeland, 1985) “[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . .” (Bellman, 1978)	Thinking Rationally “The study of mental faculties through the use of computational models.” (Charniak and McDermott, 1985) “The study of the computations that make it possible to perceive, reason, and act.” (Winston, 1992)
Acting Humanly “The art of creating machines that perform functions that require intelligence when performed by people.” (Kurzweil, 1990) “The study of how to make computers do things at which, at the moment, people are better.” (Rich and Knight, 1991)	Acting Rationally “Computational Intelligence is the study of the design of intelligent agents.” (Poole <i>et al.</i> , 1998) “AI . . . is concerned with intelligent behavior in artifacts.” (Nilsson, 1998)

Figure 1.1 Some definitions of artificial intelligence, organized into four categories.

Test Turing
NLP, KRR, ML



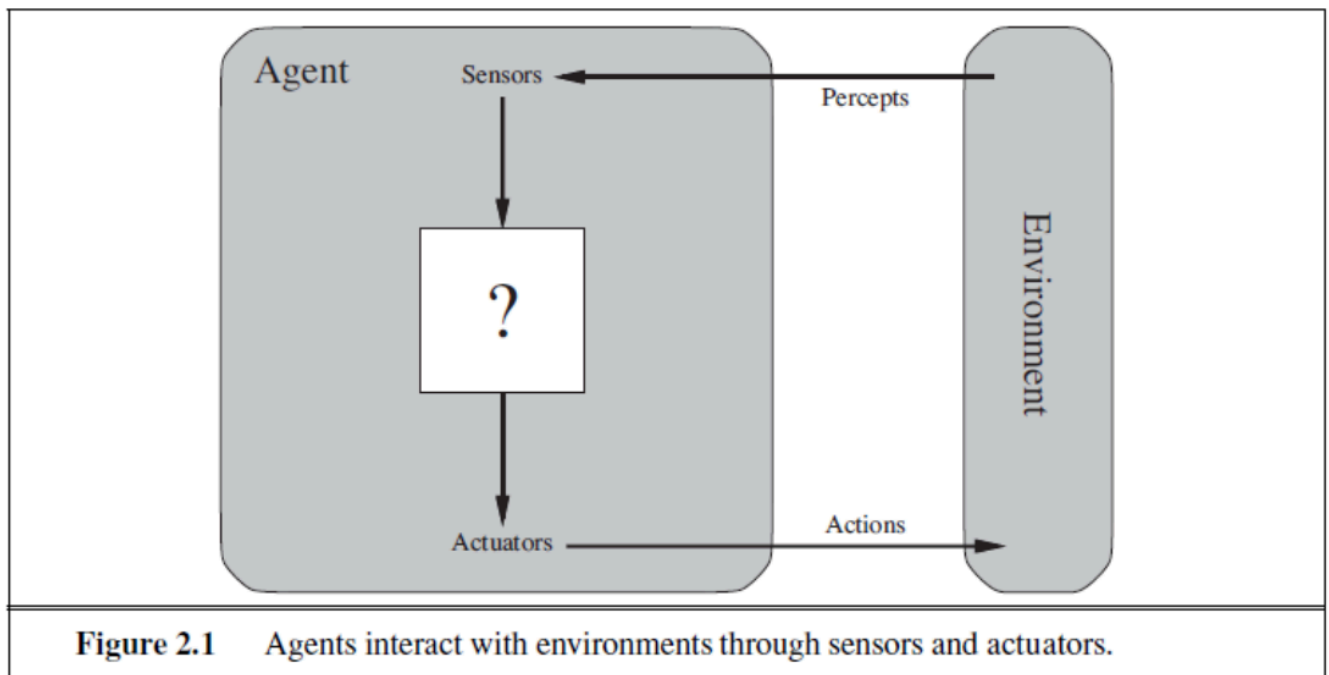
Agentes
racionales



Fuente: Russell, Norvig

1.1.2 Agentes y racionalidad

Un **agente** es cualquier cosa que **percibe** su entorno mediante **sensores** y **actúa** sobre él mediante **actuadores**. Un **agente racional** es el que actúa para obtener **el mejor resultado** o, cuando hay incertidumbre, **el mejor resultado esperado**. Este enfoque aprovecha elementos de los demás (lo que exige el test de Turing, la lógica), pero asume una **racionalidad limitada** en lugar de perfecta: no siempre se puede calcular la decisión óptima, así que a veces basta con una buena. Un ejemplo es la diferencia entre el **razonamiento lógico**, exacto, y el **razonamiento aproximado**.



El agente recibe percepciones del entorno a través de los sensores, decide (el "?") y devuelve acciones mediante los actuadores.

El ejemplo clásico es el **agente aspirador**: un mundo con dos casillas, A y B, que pueden estar sucias o limpias. El agente percibe en qué casilla está y si hay suciedad, y su comportamiento se puede describir con una tabla que asocia cada **secuencia de percepciones** con una **acción** (si está sucio, aspirar; si no, moverse a la otra casilla).

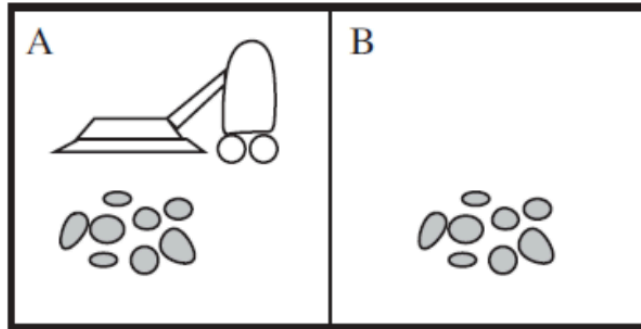


Figure 2.2 A vacuum-cleaner world with just two locations.

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
⋮	⋮
[A, Clean], [A, Clean], [A, Clean]	Right
[A, Clean], [A, Clean], [A, Dirty]	Suck
⋮	⋮

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.

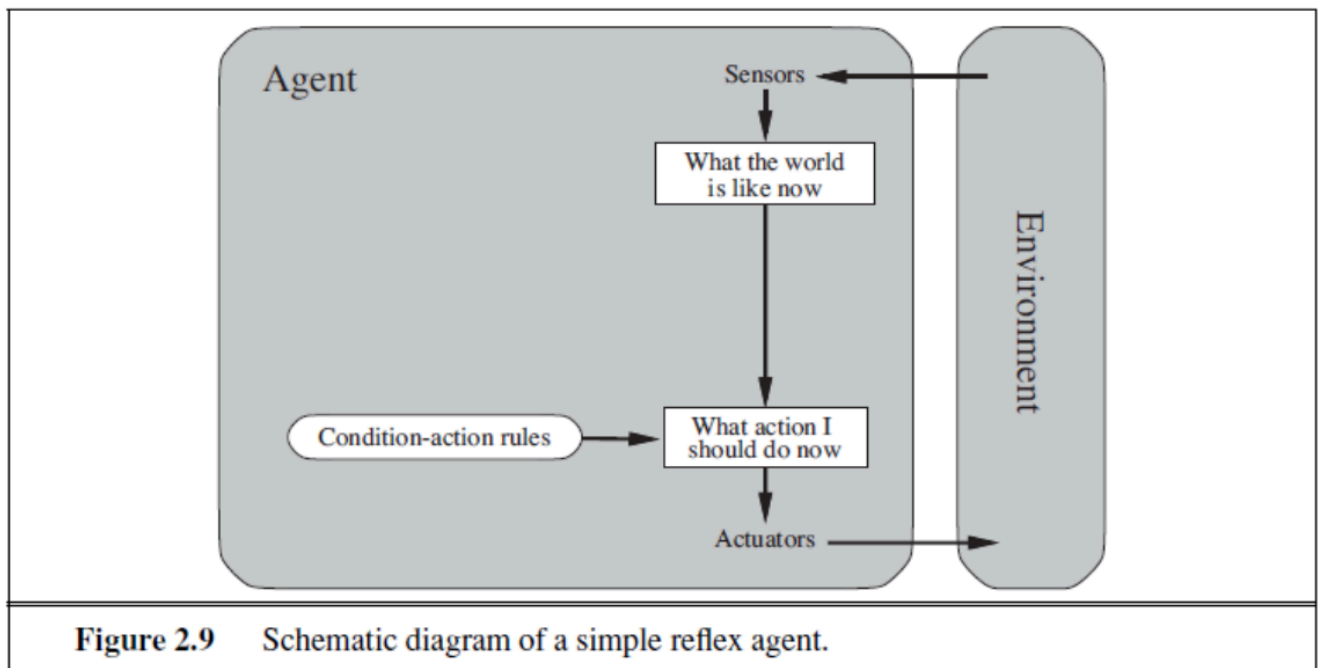
Que un agente sea racional depende de cuatro factores: la **medida de rendimiento**, que define el criterio de éxito; el **conocimiento a priori del entorno**; las **acciones** que puede realizar, y la **secuencia de percepciones** hasta el momento. Con ellos se define el agente racional:

"Para cada posible secuencia de percepciones, un agente racional debe seleccionar la acción que maximice la medida de rendimiento, teniendo en cuenta las evidencias proporcionadas por la secuencia de percepciones y cualquier conocimiento del que el agente disponga."

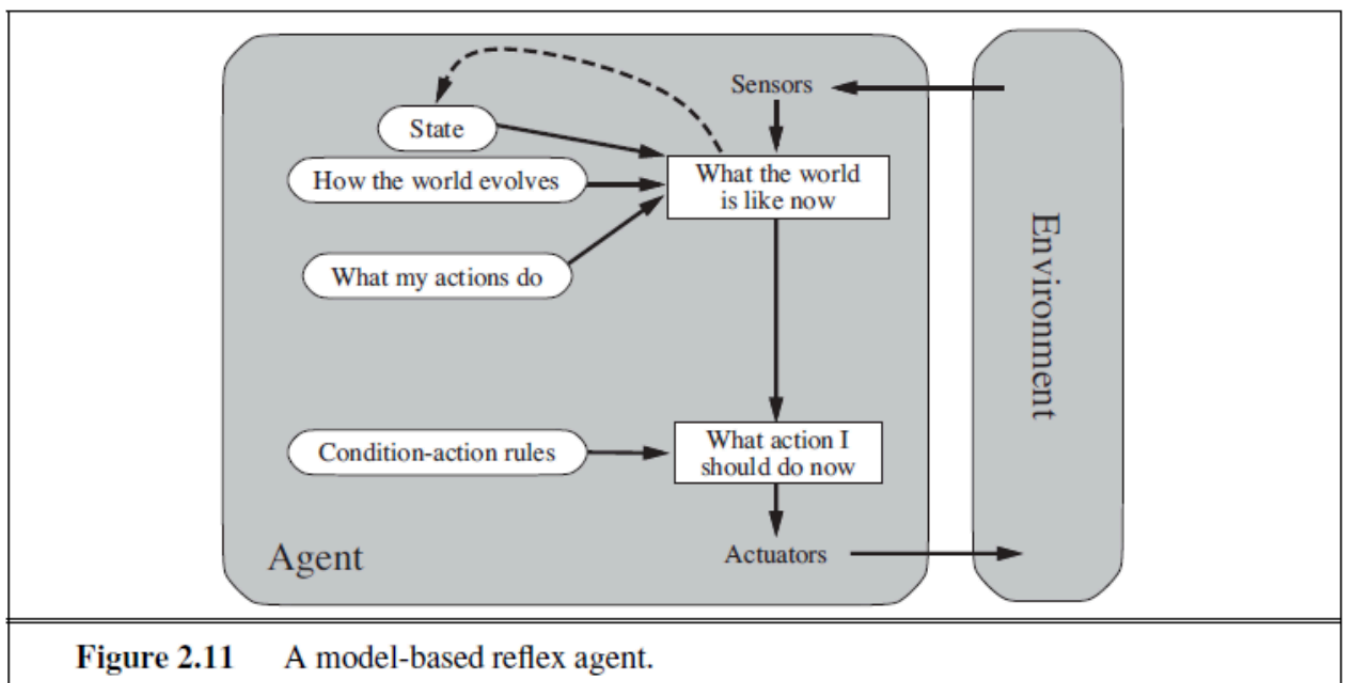
1.1.3 Tipos de agentes

La funcionalidad de un agente la implementa su **programa agente**, que se ejecuta sobre una arquitectura: **Agente = arquitectura + programa**. Hay distintos diseños, que se diferencian en **qué información usan para decidir**, en su **eficiencia y flexibilidad**, y que se eligen según **la naturaleza del entorno**.

Los **agentes reactivos** son los más simples: responden de forma **directa a la percepción instantánea** mediante reglas condición-acción ("si veo esto, hago aquello"), sin memoria.



Los **agentes reactivos basados en modelos** añaden un **modelo del entorno**: guardan un **estado interno** que resume lo que no ven en ese instante, sabiendo cómo evoluciona el mundo y qué efecto tienen sus acciones.



Los **agentes basados en objetivos** combinan la información actual con un **objetivo final**: además de saber cómo está el mundo, se preguntan "¿qué pasará si hago la acción A?" y eligen la que les acerque al objetivo.

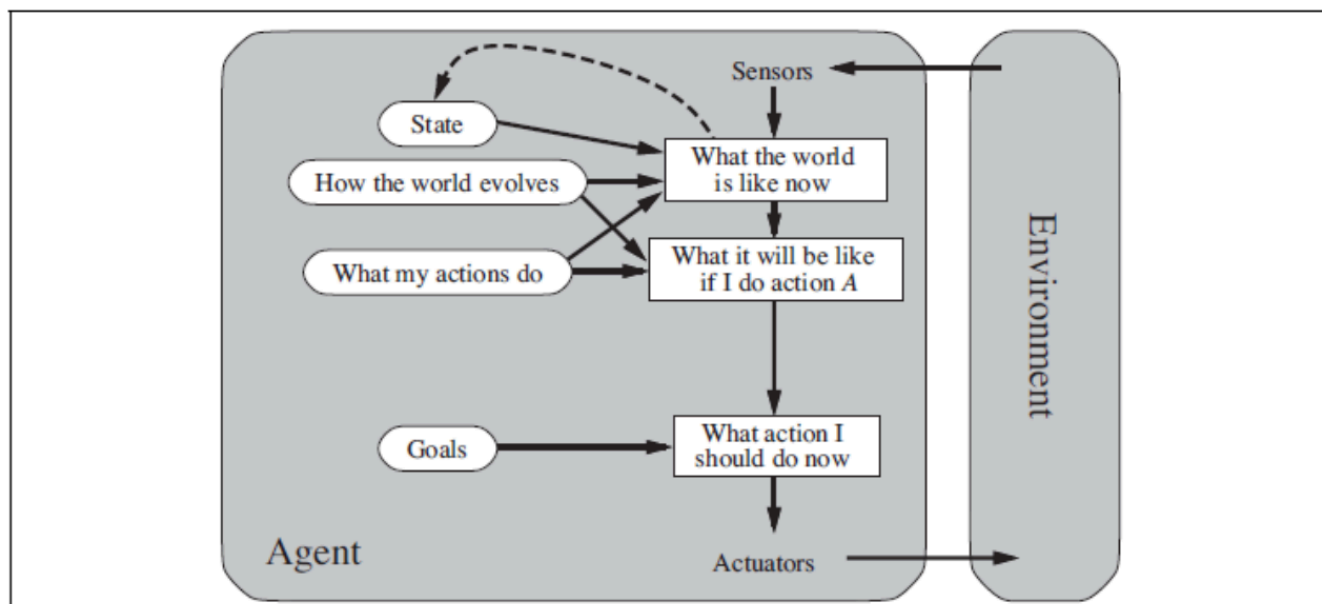


Figure 2.13 A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

Los **agentes de utilidad complementan los objetivos** con una **función de utilidad** que mide "cómo de contento estaré en ese estado". Así pueden elegir entre varias formas de alcanzar un objetivo, o decidir cuando hay incertidumbre, escogiendo la acción con mayor **utilidad esperada**.

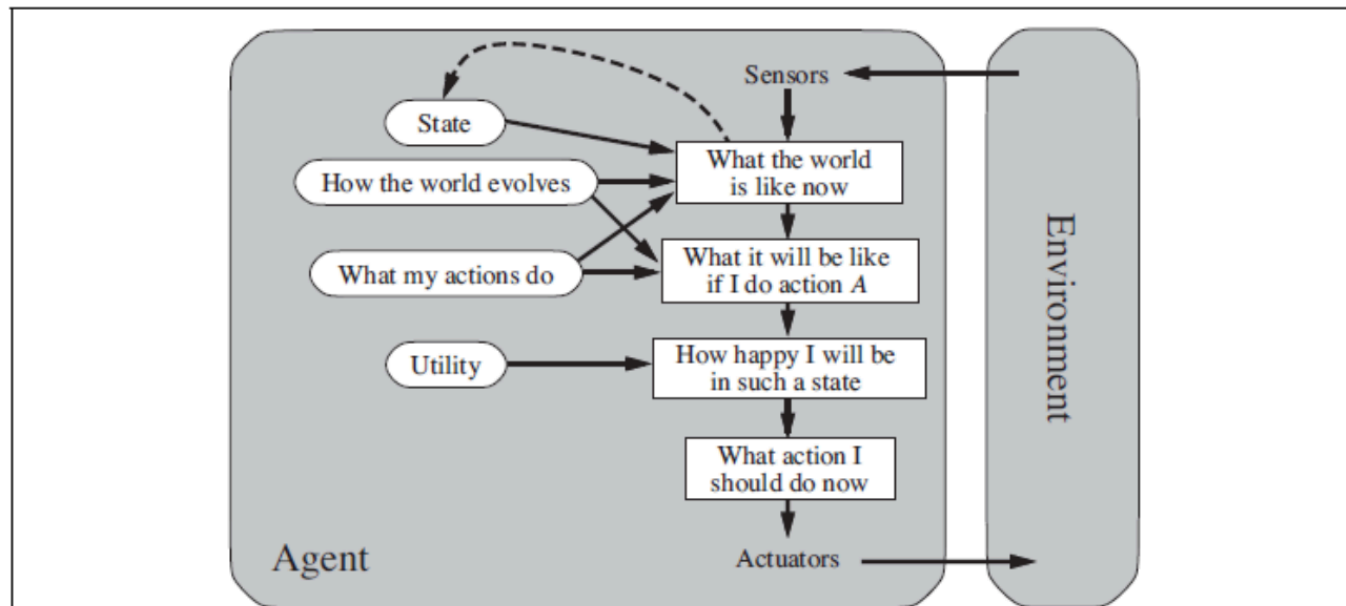


Figure 2.14 A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Cualquiera de estos agentes puede **mejorar su rendimiento mediante el aprendizaje**.

ANALOGÍA: CUATRO "CONDUCTORES"

- **Reactivo**: un termostato. Si hace frío, enciende la calefacción; no recuerda nada ni planifica.
- **Reactivo basado en modelos**: un conductor que recuerda que hay un coche en su ángulo muerto aunque ahora no lo vea.
- **Basado en objetivos**: un GPS que busca *una* ruta que llegue al destino.
- **De utilidad**: un GPS que, entre todas las rutas que llegan, elige la más rápida, sin peajes o con menos consumo, porque puntúa cada opción.

1.1.4 Agentes para la resolución de problemas

Los agentes que resuelven problemas son agentes **basados en objetivos**: de planificación, de búsqueda (no informada, informada...), etc. Para aplicarlos hay que formular el problema con estos elementos:

- Una **representación del entorno** (estados) y un **estado inicial** E_0 .
- Las **acciones** posibles (fijas).
- El **resultado** de cada acción (**modelo de transiciones**).
- Una **prueba de finalización** que reconozca el **estado objetivo** E_f .
- Una **función de coste**.

Una **solución** es una **secuencia de acciones que lleva de E_0 a E_f** , y la solución **óptima** es la de **menor coste**.

1.2 Búsqueda local

1.2.1 Entornos y óptimos locales

La búsqueda local trabaja sobre el **espacio de soluciones** S y necesita definir, para cada solución, cuáles son sus "vecinas". Eso lo hace una **estructura de entornos**, una aplicación N que asocia a cada solución s su **entorno o vecindario** $N(s)$:

$$N : S \rightarrow \wp(S), \quad s \mapsto N(s) \subseteq S$$

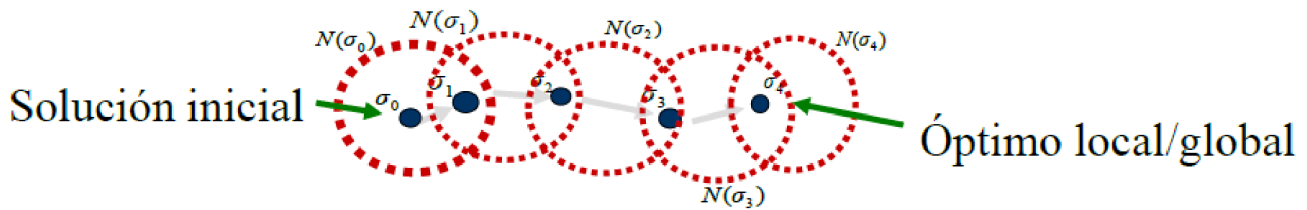
($\wp(S)$)

Con esto se define un **mínimo local**: una solución s' es mínimo local respecto a N si ningún vecino suyo es mejor,

$$f(s') \leq f(s) \quad \forall s \in N(s') \subseteq S$$

El término **local** se refiere precisamente a que solo se mira el **entorno próximo** de la solución. A partir de una solución s se generan **soluciones vecinas** con un **operador de generación**, y la metaheurística

estudia localmente ese entorno para decidir por dónde seguir. Una **búsqueda local** es, por tanto, un proceso que, dada la solución actual, **selecciona iterativamente una solución de su entorno** para continuar.



Desde la solución inicial σ_0 se salta a una vecina dentro de su entorno $N(\sigma_0)$, desde ella a otra de $N(\sigma_1)$, y así hasta llegar a σ_4 , un óptimo local (que puede ser o no el global).

ANALOGÍA: EXCURSIONISTA EN LA NIEBLA

Imagina que buscas el punto más bajo de una cordillera en plena niebla: solo ves el suelo a un paso de tus pies. El **entorno** son los sitios a los que puedes llegar dando un paso, y la **búsqueda local** consiste en ir dando pasos cuesta abajo. El problema es que, al llegar al fondo de un valle, todos los pasos posibles son cuesta arriba y crees haber terminado, aunque al otro lado de la montaña haya un valle mucho más profundo. Ese valle es un **mínimo local**; el más profundo de todos, el **mínimo global**.

Ejemplo que usaremos en todo el tema

Para ver los algoritmos funcionando usaremos un problema de juguete: hay 10 soluciones posibles, numeradas del 1 al 10, cada una con un coste que queremos **minimizar**. El **entorno** de una solución son sus dos vecinas (la de la izquierda y la de la derecha).

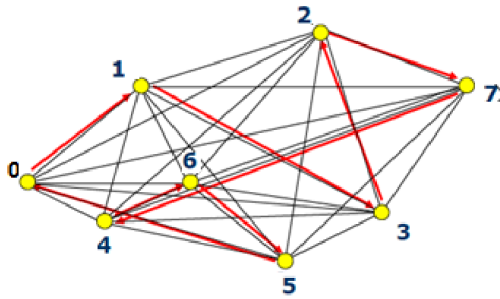
Solución x	1	2	3	4	5	6	7	8	9	10
Coste $C(x)$	9	7	5	8	10	7	4	1	3	6

La solución **3** es un **mínimo local** (sus vecinas cuestan 7 y 8, ambas peores) y la **8** es el **mínimo global** (coste 1). Entre ambas hay una "montaña" (las soluciones 4 y 5).

1.2.2 Operadores de entornos

Los operadores definen cómo se genera un vecino. Se ilustran con el **problema del viajante (TSP)**, donde una solución es el orden en que se visitan las ciudades (una ruta de 0 a 0 pasando por todas).

La **inserción** extrae un elemento y lo **inserta en otra posición**. En el ejemplo, de la ruta 0-1-3-2-7-4-6-5-0 se saca la ciudad 4 y se coloca tras la 1, dando 0-1-4-3-2-7-6-5-0.

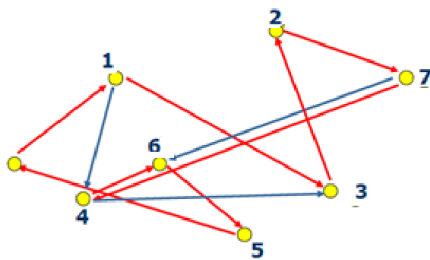


	0	1	2	3	4	5	6	
0	1	3	2	7	4	6	5	0

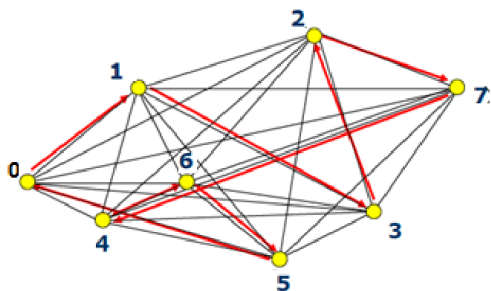
	0		1	2	3	4	5	6	
0	1		3	2	7	4	6	5	0



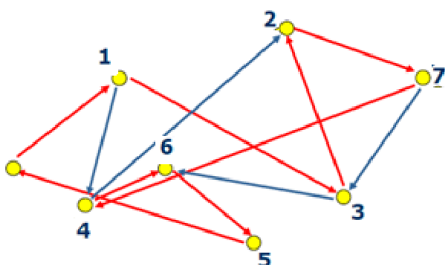
	0	1	2	3	4	5	6	
0	<u>1</u>	<u>4</u>	<u>3</u>	2	<u>7</u>	<u>6</u>	5	0



El **intercambio** escoge **dos elementos y los intercambia** de posición: intercambiando el 3 y el 4, la ruta pasa a 0-1-4-2-7-3-6-5-0.

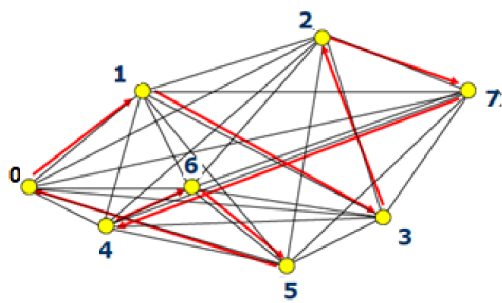


	0	1	2	3	4	5	6	
0	1	3	2	7	4	6	5	0

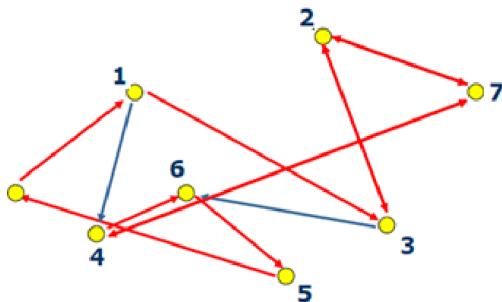


	0	1	2	3	4	5	6	
0	<u>1</u>	<u>4</u>	<u>2</u>	<u>7</u>	<u>3</u>	<u>6</u>	5	0

La **inversión simple** escoge una **sublista y le da la vuelta**: invirtiendo el tramo 3-2-7-4 se obtiene 0-1-4-7-2-3-6-5-0.



	0	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	5	6	
0	1	<u>3</u>	<u>2</u>	<u>7</u>	<u>4</u>	6	5	0



	0	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	5	6	
0	1	<u>4</u>	<u>7</u>	<u>2</u>	<u>3</u>	6	5	0

El operador elegido decide **cuántos vecinos tiene cada solución** y, con ello, cómo se comporta la búsqueda. Un entorno **pequeño** (pocos vecinos) es barato de explorar en cada paso, pero hay más soluciones que parecen mínimos locales porque "se ve poco". Un entorno **grande** hace que haya menos óptimos locales en los que quedarse atascado, pero cada paso cuesta más, porque hay que evaluar más vecinos. En nuestro ejemplo de juguete, si el entorno fueran las **4** soluciones más cercanas (dos a cada lado), desde la 3 se "vería" la 5 y la 1, pero todavía no la 7 ni la 8.

1.2.3 Búsqueda local iterativa

El esquema general de búsqueda por entornos parte de una solución inicial y, en cada paso, genera un vecino, decide si **acepta** moverse a él y guarda la mejor solución vista, hasta cumplir un criterio de parada:

Procedimiento Búsqueda por Entornos

Inicio

GENERA(Solución Inicial)

Solución Actual ← Solución Inicial

Mejor Solución ← Solución Actual

Repetir

Solución Vecina ← GENERA_VECINO(Solución Actual)

Si Acepta(Solución Vecina) entonces Solución Actual ← Solución Vecina

Si Objetivo(Solución Actual) es mejor que Objetivo(Mejor Solución)
entonces Mejor Solución ← Solución Actual

Hasta (Criterio de parada)

DEVOLVER (Mejor Solución)

Fin

La variante más sencilla es la estrategia "**el mejor**": en cada paso se examinan **todos** los vecinos, se elige el mejor y solo se avanza si mejora a la solución actual; en cuanto ningún vecino mejora, se para. Es una aproximación **voraz y monótona**: siempre va a mejor, por lo que se queda atrapada en el **primer óptimo local** que encuentra.

En el ejemplo, si empezamos en $x = 2$ (coste 7), la búsqueda del mejor mira las vecinas 1 (coste 9) y 3 (coste 5), y se mueve a la 3. Desde ahí mira la 2 (coste 7) y la 4 (coste 8): ninguna mejora, así que **se para en la 3 con coste 5**, sin enterarse de que la 8 cuesta 1. Si hubiéramos empezado en $x = 6$, habría bajado a la 7 y luego a la 8 y habría encontrado el óptimo: **el resultado depende totalmente de dónde se empiece**.

Procedimiento Búsqueda Local del Mejor

Inicio

 GENERA(S_act)

 Repetir

 Mejor Vecino $\leftarrow$ S_act

 Repetir para toda $S' \in E(S_{act})$

$S' \leftarrow$ GENERA_VECINO(S_act)

 Si Objetivo(S') mejor que Objetivo(Mejor Vecino) entonces Mejor Vecino $\leftarrow S'$

 Fin-Repetir-para

 Si Objetivo(Mejor Vecino) mejor que Objetivo(S_act) entonces S_act $\leftarrow$ Mejor

Vecino

 Hasta (Objetivo(Mejor Vecino) peor o igual que Objetivo(S_act))

 DEVOLVER(S_act)

Fin

PARA ENTENDERLO MEJOR

[Búsqueda local. Subir la Colina](#) — el algoritmo de "subir la colina" y por qué se queda en óptimos locales (vídeo, 5:30).

1.3 Metaheurísticas

1.3.1 De la heurística a la metaheurística

Una **heurística** resuelve problemas **incorporando conocimiento experto** de un dominio, es decir, resuelve "inteligentemente" usando ese conocimiento, y ofrece un buen rendimiento tanto en **calidad de las soluciones** como en **recursos empleados**. Puede ser **específica**, diseñada *ad hoc* para un

problema concreto y con el **rendimiento más alto**, o **general**, basada en **estrategias generales** que son **más sencillas, adaptables y robustas**.

Las **metaheurísticas** van "más allá" de la heurística: son abstracciones o técnicas generales que producen **soluciones factibles o subóptimas**. Su definición clásica es de **Glover (1986)**: *estrategias inteligentes para diseñar o mejorar procedimientos heurísticos generales con un alto rendimiento*.

1.3.2 Problemas de optimización

Optimizar es buscar **soluciones factibles** y, entre ellas, la **óptima** según uno o varios **criterios u objetivos**. Formalmente, un **problema de optimización** consiste en encontrar, dentro del conjunto S de soluciones factibles (el **espacio de soluciones**), la que optimiza una **función objetivo** $f(s)$. Si se trata de minimizar, se escribe

$$\min\{f(s) \mid s \in S\}$$

y una **solución óptima** s^* o **mínimo global** es aquella en la que se alcanza ese mínimo: $f(s^*) \leq f(s) \quad \forall s \in S$.

Ejemplos reales son el **camino más corto** entre varios puntos, el plan de **reparto de mercancías** de mínimo coste, la **asignación óptima de trabajadores a tareas**, la **secuencia óptima de trabajos** en una cadena de producción, la distribución de **tripulaciones de aviones** con mínimo coste o el mejor **enrutamiento de paquetes** en Internet. El ejemplo clásico es el **TSP** (problema del viajante): encontrar la ruta más corta que visite todas las ciudades una vez y vuelva al origen.

Estos problemas suelen consistir en **agrupar, ordenar o asignar** un conjunto discreto de objetos que cumplan **ciertas restricciones**, aparecen en muchísimas áreas y tienen una **gran complejidad computacional (son NP-duros)**. Por eso los **algoritmos exactos** (programación dinámica, backtracking...) son **ineficientes o directamente imposibles** de aplicar, y en la práctica se usan **algoritmos aproximados** que dan **buenas soluciones**, no necesariamente la óptima, **en un tiempo razonable**.

1.3.3 Algoritmos aproximados

Los **algoritmos aproximados** dan soluciones **cercanas a la óptima** en problemas NP-duros en un tiempo razonable. Interesan cuando **no hay un método exacto**, o cuando el que hay necesita demasiado tiempo y memoria, y cuando **no hace falta la solución óptima** porque basta con una **buena calidad en un tiempo aceptable**.

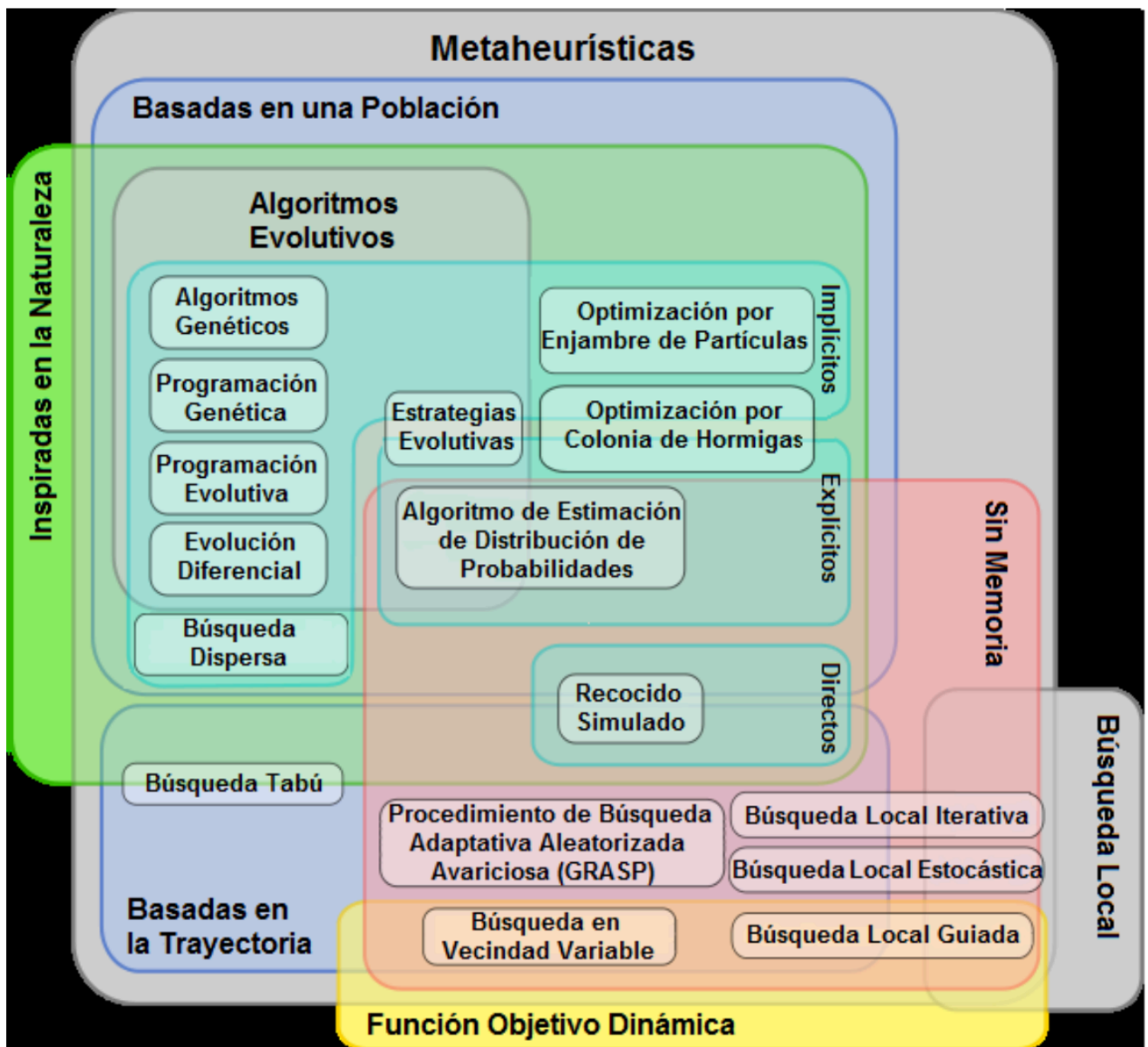
1.3.4 Qué son y cómo se clasifican

Las **metaheurísticas** son una familia de **algoritmos aproximados de propósito general**. Suelen ser **procedimientos iterativos** que **guían una heurística subordinada de búsqueda**, combinando de

forma inteligente distintos conceptos para **explorar** y **explotar** adecuadamente el espacio de búsqueda.

Ventajas	Inconvenientes
Algoritmos de propósito general	Son aproximados , no exactos
Gran éxito en la práctica	Son no deterministas (probabilísticos) : dos ejecuciones pueden dar resultados distintos
Fáciles de implementar y paralelizar	No siempre tienen una base teórica establecida

Se pueden caracterizar por su **naturaleza**: si siguen una **trayectoria** (una sola solución que va cambiando) o trabajan con **poblaciones** de soluciones, si usan **memoria**, y en qué se **inspiran** (biología, comportamiento animal, física). El tema se centra en las **metaheurísticas de búsqueda o trayectorias**, que recorren el espacio de soluciones **transformando iterativamente una solución de partida** (es decir, hacen búsqueda local). Ejemplos: el **Enfriamiento Simulado (SA)** y la **Búsqueda Tabú (TS)**.



Clasificación (Wikipedia): basadas en población (algoritmos evolutivos, enjambre de partículas, colonia de hormigas...) frente a basadas en trayectoria (búsqueda tabú, GRASP, búsqueda en vecindad variable...), inspiradas o no en la naturaleza, con o sin memoria. El recocido simulado está inspirado en la naturaleza y no usa memoria.

PARA ENTENDERLO MEJOR

[Metaheurística \(Wikipedia\)](#) — definición, clasificación y fuente de la figura anterior (artículo; enlazado en las diapositivas). [Metaheurísticas — Fernando Sancho \(Univ. de Sevilla\)](#) — introducción clara a búsqueda local y metaheurísticas (artículo; referencia complementaria del PDF).

1.3.5 Intensificación y diversificación

El funcionamiento de una metaheurística es un equilibrio entre dos fuerzas opuestas. La **intensificación** es el esfuerzo dedicado a buscar **en la región actual** (**explotación** del espacio), y la **diversificación**, el esfuerzo dedicado a buscar **en regiones lejanas** (**exploración**). Hace falta equilibrarlas para **identificar rápido las regiones con soluciones de buena calidad** sin **perder tiempo en regiones poco prometedoras o ya exploradas**.

ANALOGÍA: BUSCAR DÓNDE CENAR EN UNA CIUDAD NUEVA

Diversificar es recorrer barrios distintos para descubrir dónde están los buenos restaurantes; **intensificar** es, una vez encontrada una calle prometedora, probar los locales de esa calle. Si solo diversificas, pasas la noche caminando sin cenar; si solo intensificas, te quedas en la primera calle aceptable aunque la mejor zona esté a dos manzanas. Lo sensato es explorar mucho al principio y centrarse al final, que es exactamente lo que hace el temple simulado.

1.4 Temple Simulado (SA)

1.4.1 Idea

El **enfriamiento, temple o recocido simulado** (*Simulated Annealing*, **SA**) es un algoritmo de **búsqueda por entornos** con un **criterio probabilístico de aceptación de soluciones** inspirado en la **termodinámica**. Lo popularizaron Kirkpatrick, Gelatt y Vecchi en *Optimization by Simulated Annealing* (Science, 1983).

Su idea central es evitar que la búsqueda local se quede atrapada en **óptimos locales** permitiendo **movimientos hacia soluciones peores**. Pero si la búsqueda está progresando hacia una buena solución, esos movimientos "de escape" tienen que estar **controlados**. El SA lo consigue con una **función de probabilidad** que hace que aceptar soluciones peores sea **cada vez menos probable a medida que avanza la búsqueda**. Es la filosofía habitual: **diversificar al principio e intensificar al final**.

ANALOGÍA 1: EL HERRERO

Para que el metal quede fuerte y sin defectos, el herrero lo calienta y lo **enfría despacio**. Con el metal caliente, los átomos se mueven mucho y pueden recolocarse libremente; al ir enfriando, se mueven cada vez menos hasta quedar fijos en una estructura ordenada, de **energía mínima**. Si lo enfría de golpe (lo mete en agua fría), los átomos se quedan "congelados" en una estructura desordenada: un **mínimo local**.

ANALOGÍA 2: LA CANICA EN UNA BANDEJA CON HOYOS

Pon una canica en una bandeja llena de hoyos de distinta profundidad y quieres que acabe en el más hondo. Si solo la dejas rodar, se queda en el primer hoyo (**búsqueda local**). Si **agitas la bandeja con fuerza** al principio, la canica salta de hoyo en hoyo; si luego **agitas cada vez más suave**, llega un momento en que ya no puede salir de los hoyos profundos, pero sí de los poco profundos. Al final, con poca agitación, se queda en uno de los más hondos. La **fuerza de la agitación es la temperatura**.

PARA ENTENDERLO MEJOR

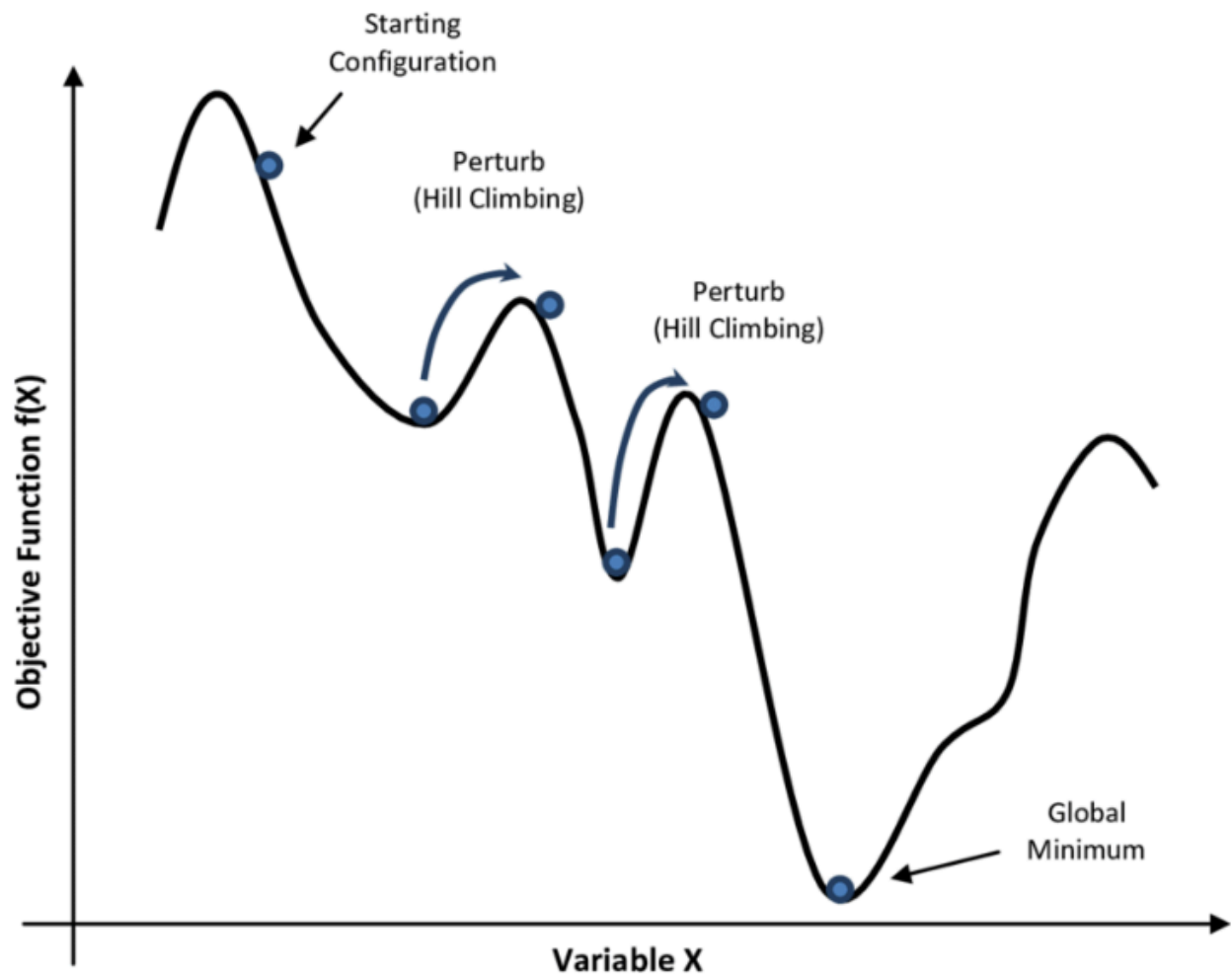
[Recocido simulado \(3 de 8\). Recocido Simulado](#) — cómo se traslada la idea del recocido a la optimización (vídeo, 7 min; José Manuel Galán, serie de 8 vídeos).

1.4.2 Origen termodinámico

N. Metropolis (1953) modeló el enfriamiento de un material simulando los cambios de energía de un sistema de partículas al bajar la temperatura, hasta que converge a un **estado estable (congelado)**. Según la termodinámica, a una temperatura T , la probabilidad de un **incremento de energía** de magnitud δE es aproximadamente

$$P[\delta E] = e^{-\frac{\delta E}{kT}} = \frac{1}{e^{\frac{\delta E}{kT}}}$$

donde k es la **constante de Boltzmann**. La simulación genera una **perturbación aleatoria** y calcula el cambio de energía: si la energía **baja**, el cambio se **acepta siempre**; si **sube**, se acepta **con la probabilidad anterior**.



Fuente: F. Sancho

Una búsqueda que solo baja (hill climbing) se queda en el primer valle que encuentra; aceptar de vez en cuando "subidas" permite saltar a otros valles y llegar al mínimo global.

La analogía entre ambos mundos es directa:

Simulación termodinámica	Optimización combinatoria
Estados del sistema	Soluciones factibles
Energía	Coste
Cambio de estado	Solución en el entorno
Temperatura	Parámetro de control
Estado congelado	Solución heurística

El algoritmo usa por tanto una variable **temperatura T** que determina **cuánto se aceptan soluciones vecinas peores**. Empieza en un valor alto, la **temperatura inicial T_0** , y se va reduciendo en cada iteración mediante un **mecanismo de enfriamiento α** hasta llegar a una **temperatura final T_f** .

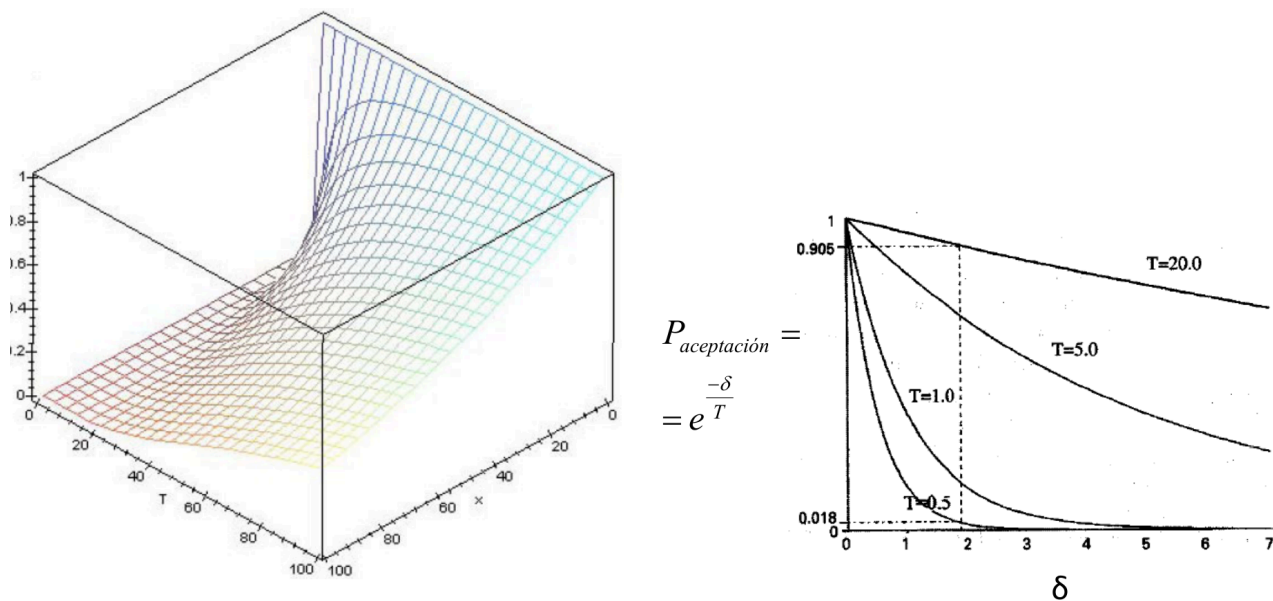
1.4.3 Criterio de aceptación

En cada iteración se generan **L(T)** soluciones candidatas, un número que puede ser **fijo** en toda la ejecución (incluso 1) o **depender de la iteración**. A cada candidata se le aplica el **criterio de aceptación**: si es **mejor** que la actual, se **acepta automáticamente**, como en la búsqueda local clásica; si es **peor**, se acepta **con cierta probabilidad**, y eso es lo que permite salir de los óptimos locales en los que la búsqueda clásica se quedaría atascada.

Esa probabilidad depende de la **diferencia de costes** δ entre la vecina y la actual, y de la temperatura:

$$\delta = C(s_{vecina}) - C(s_{actual}), \quad P_{aceptación} = e^{-\delta/T}$$

Si $\delta < 0$ el vecino es **mejor** (cuesta menos) y si $\delta > 0$ es **peor**. De la fórmula salen dos consecuencias. Primero, **a mayor temperatura, mayor probabilidad** de aceptar soluciones peores, así que al principio se aceptan soluciones mucho peores (**exploración**) y al final casi ninguna (**explotación**). Segundo, **a menor diferencia de costes, mayor probabilidad** de aceptación: se aceptan con más facilidad las soluciones "no mucho peores".



Izquierda: superficie de $P = e^{-\delta/T}$ según la diferencia de coste y la temperatura. Derecha: curvas para $T = 20, 5, 1$ y $0,5$.

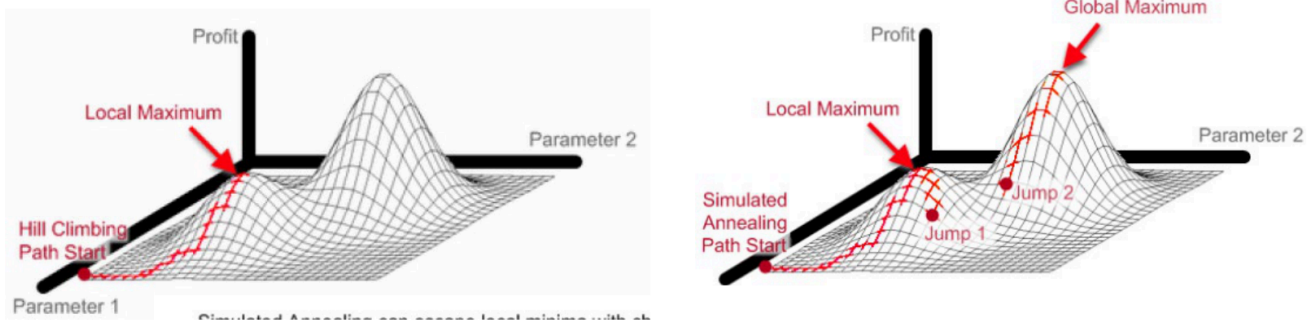
Por ejemplo, para una diferencia fija $\delta = 2$, la probabilidad de aceptar el movimiento es $e^{-2/20} = 0,905$ con $T = 20$, pero $e^{-2/0,5} = 0,018$ con $T = 0,5$: se reduce en un factor $0,905/0,018 \approx 50$.

La tabla siguiente muestra el efecto combinado de las dos variables. Cada casilla es la probabilidad de aceptar una solución **peor**, según cuánto peor sea (δ) y la temperatura (T):

	$\delta = 1$ (un poco peor)	$\delta = 5$	$\delta = 10$ (mucho peor)
$T = 20$ (caliente)	0,95	0,78	0,61
$T = 5$	0,82	0,37	0,14

	$\delta = 1$ (un poco peor)	$\delta = 5$	$\delta = 10$ (mucho peor)
$T = 1$ (frío)	0,37	0,007	0,00005

Leyendo **por filas**: a igual temperatura, cuanto peor es la solución, menos se acepta. Leyendo **por columnas**: al enfriar, todo se acepta menos, y lo que antes era casi seguro (0,61 para un empeoramiento de 10) pasa a ser prácticamente imposible. Un truco útil: cuando $\delta = T$, la probabilidad es siempre $e^{-1} \approx 0,37$. Por eso la temperatura se puede entender como "**el tamaño de empeoramiento que todavía se acepta con cierta frecuencia**".



Maximizando: el hill climbing (izquierda) sube hasta el máximo local más cercano y se para; el recocido simulado (derecha) da "saltos" que le permiten pasar al máximo global.

1.4.4 El algoritmo básico

Cada iteración genera $L(T)$ vecinos aplicando el criterio de aceptación y, al terminarla, **se enfría la temperatura** y se pasa a la siguiente. El esquema para **minimizar el coste** es:

```

INPUT (T0, α, L, Tf)
T ← T0
S_act ← Genera_solucion_inicial
WHILE T ≥ Tf DO
  FOR cont ← 1 TO L(T) DO
    S_cand ← Selecciona_solucion_N(S_act)
    δ ← coste(S_cand) - coste(S_act)
    IF (U(0,1) < e-(δ/T)) OR (δ < 0) THEN S_act ← S_cand
  END
  T ← α(T)
END
{Escribe como solución la mejor de las S_act visitadas}

```

$U(0,1)$ es un número aleatorio uniforme entre 0 y 1: comparar con él es la forma de "aceptar con probabilidad $e^{-\delta/T}$ ". Si el objetivo fuera **maximizar**, las desigualdades irían en sentido contrario.

Las variables del algoritmo

Variable	Qué es	En la analogía de la bandeja
T	Temperatura actual: cuánto se toleran las soluciones peores	Fuerza con la que agitas ahora
T_0	Temperatura inicial (alta)	Agitación del principio
T_f	Temperatura final: cuando $T < T_f$ se para	Agitación mínima a partir de la cual paras
$\alpha(T)$	Mecanismo de enfriamiento: cómo baja T (p. ej. $T \leftarrow 0,9 \cdot T$)	Lo rápido que suavizas la agitación
$L(T)$	Nº de vecinos que se prueban a cada temperatura	Cuánto rato agitas antes de suavizar
S_{act}	Solución actual	Hoyo en el que está la canica
S_{cand}	Candidata: vecina elegida al azar de $N(S_{act})$	Hoyo al que podría saltar
δ	$coste(S_{cand}) - coste(S_{act})$; negativo = mejora	Si el nuevo hoyo es más hondo ($\delta < 0$) o menos ($\delta > 0$)
$U(0, 1)$	Número aleatorio entre 0 y 1: "tirar el dado"	El azar de cada sacudida

Un detalle importante: el algoritmo devuelve **la mejor solución visitada**, no la última, porque al aceptar soluciones peores puede terminar en una peor que otra que ya pasó.

Ejemplo paso a paso

Aplicamos el algoritmo al ejemplo de juguete, empezando en el mínimo local $x = 3$ (coste 5) donde se atascaba la búsqueda local, con $T_0 = 10$, $\alpha(T) = 0,5 \cdot T$, $L(T) = 2$ vecinos por temperatura y $T_f = 1$. Los valores de U son los "dados" que han salido.

T	Actual $\rightarrow$ candidata	δ	$P = e^{-\delta/T}$	U	¿Se acepta?
10	3 (5) $\rightarrow$ 4 (8)	+3	0,741	0,52	Sí: $0,52 < 0,741$, sube la "montaña"
10	4 (8) $\rightarrow$ 5 (10)	+2	0,819	0,33	Sí: llega a la cima
5	5 (10) $\rightarrow$ 6 (7)	-3	—	—	Sí: mejora
5	6 (7) $\rightarrow$ 7 (4)	-3	—	—	Sí: mejora
2,5	7 (4) $\rightarrow$ 8 (1)	-3	—	—	Sí: llega al óptimo global
2,5	8 (1) $\rightarrow$ 9 (3)	+2	0,449	0,60	No
1,25	8 (1) $\rightarrow$ 7 (4)	+3	0,091	0,65	No
1,25	8 (1) $\rightarrow$ 9 (3)	+2	0,202	0,40	No

Tras enfriar a $T = 0,625 < T_f$, el algoritmo se para y devuelve la solución 8, con coste 1. Fíjate en la dinámica: **con T alta se permite subir la montaña** que separaba los dos valles; **con T baja ya no se**

aceptan empeoramientos y el algoritmo se queda en el valle bueno. Si en la fila de $T = 2,5$ el dado hubiera dado $U = 0,30$, se habría movido a la 9: por eso se guarda la mejor solución visitada.

¿Y si eligiéramos mal T_0 ? Con $T_0 = 0,5$, la probabilidad de dar el primer paso de 3 a 4 sería $e^{-3/0,5} = 0,0025$: prácticamente nunca saldría del valle y el SA se comportaría como la búsqueda local. Con $T_0 = 1000$, esa probabilidad sería 0,997 y se aceptaría casi todo: al principio el algoritmo daría vueltas al azar por todo el espacio, desperdiciando iteraciones hasta que la temperatura bajase lo suficiente.

Las diapositivas muestran también las versiones de **Palma y Marín** (igual que la anterior, con $\Delta E = \text{Coste}(S_1) - \text{Coste}(S)$) y de **Russell y Norvig**, que usa una función `schedule(t)` para fijar la temperatura en cada paso, termina cuando $T = 0$ y está escrita para **maximizar**: acepta si $\Delta E = \text{next.VALUE} - \text{current.VALUE} > 0$ y, si no, con probabilidad $e^{\Delta E/T}$ (que es lo mismo, porque ahí ΔE es negativo).

PARA ENTENDERLO MEJOR

[Simulated Annealing Visualization: Solving Travelling Salesman Problem](#) — el SA resolviendo un TSP: se ve cómo la ruta cambia mucho al principio y se estabiliza al enfriarse (vídeo, 3 min, inglés).

1.5 Diseño del algoritmo SA

Para aplicar el SA a un problema concreto hay que decidir varias cosas, que el pseudocódigo señala: la **representación** de las soluciones, la **inicialización** (temperatura y solución inicial), la **transición** entre soluciones y la **secuencia de enfriamiento**.

1.5.1 Representación, inicialización y transiciones

La **representación** depende del problema: un **vector ordenado de enteros** para el TSP (representación de orden), un **vector binario** $\{0, 1\}$ para la mochila (cada objeto entra o no), o un **vector de reales** para optimización continua. La **inicialización**, tanto de la **temperatura inicial** como de la **solución inicial**, conviene hacerla con **técnicas eficientes y conocimiento** del problema.

Dentro del bucle, la **transición de soluciones** tiene tres pasos: **generar una nueva solución** del entorno (S_{cand}), **calcular la diferencia de costes** δ y **aplicar el criterio de aceptación**.

La **secuencia de enfriamiento** la definen **cuatro parámetros**: el **valor inicial** de la temperatura (T_0), la **condición de parada** ($T \geq T_f$), la **velocidad de enfriamiento** $L(T)$ y el **mecanismo de enfriamiento** $\alpha(T)$. Los siguientes apartados los tratan uno a uno.

1.5.2 Temperatura inicial T_0

No conviene usar un valor fijo independiente del problema: **si es muy baja, no se alcanza el óptimo** (el algoritmo apenas explora) y **si es muy alta, tarda mucho en converger**. La idea es que al principio se acepte **casi cualquier transición**, lo que se puede ajustar **experimentalmente** o de forma **proporcional al tamaño del problema** o del **espacio de soluciones**. En general se busca que

$$\frac{\text{Núm. movimientos aceptados en } T_0}{\text{Núm. total de movimientos}} \approx 1, \quad T_0 \gg \delta$$

Una forma práctica es calcularla a partir de la **probabilidad Φ** de aceptar una solución un **μ peor** que la inicial S_0 :

$$T_0 = \frac{\mu}{-\ln(\phi)} \cdot C(S_0)$$

Deducción de la fórmula

En el algoritmo, una candidata peor ($\delta > 0$) se acepta si $U(0, 1) < e^{-\delta/T}$. La primera vez que se hace esa comparación, $T = T_0$ y la solución actual es S_0 , así que la condición límite es $U(0, 1) = e^{-\delta/T_0}$ con $\delta = C(S_{vecina}) - C(S_0)$.

Si μ es el **tanto por uno de empeoramiento** que estamos dispuestos a aceptar con probabilidad Φ , la vecina cuesta $C(S_{vecina}) = (1 + \mu) \cdot C(S_0)$, luego $\delta = \mu \cdot C(S_0)$. Sustituyendo en la condición límite:

$$\Phi = e^{-\mu \cdot C(S_0)/T_0} \Rightarrow \ln(\Phi) = -\frac{\mu}{T_0} C(S_0) \Rightarrow T_0 = \frac{-\mu}{\ln(\Phi)} C(S_0)$$

EN LA DIAPOSITIVA SE HABLA DE UNA SOLUCIÓN "UN M% PEOR", PERO EN LA DEDUCCIÓN M ES UN **TANTO POR UNO** (M = 5 SIGNIFICA UN 500% PEOR).

Ejemplo

Con $\mu = 5$ y $\Phi = 0,99$ se acepta con un 99% de probabilidad una solución $1 + 5 = 6$ veces peor que la actual. Si $C(S_0) = 100$, se aceptarían con un 99% de probabilidad soluciones de coste 600, y

$$T_0 = \frac{-5}{\ln(0,99)} \cdot 100 = \frac{-500}{\ln(0,99)} = 49749,58$$

Con esa T_0 , el umbral $e^{-\delta/T_0}$ para distintas candidatas queda así:

Coste de la candidata $C(S_n)$	δ	Probabilidad de aceptarla
100	0	1
600	500	0,9900
5.000	4.900	0,9062

Coste de la candidata $C(S_n)$	δ	Probabilidad de aceptarla
50.000	49.900	0,3668

También se puede hacer al revés, calculando qué coste tendría una solución aceptada con una probabilidad dada: $\delta = -T_0 \cdot \ln(U)$, es decir, $C(S_{vecina}) = C(S_0) - T_0 \cdot \ln(U)$.

Probabilidad U	Coste de la solución
0,2	80.169
0,1	114.652
0,05	149.136
0,001	343.758

Es decir, al principio el algoritmo acepta incluso soluciones miles de veces peores, aunque con poca probabilidad: justo la libertad inicial que se buscaba.

Cómo influyen μ y ϕ

La fórmula se entiende mejor si se lee como una frase: "*quiero que, al principio, una solución un μ peor que la inicial se acepte con probabilidad ϕ* ". Subir cualquiera de los dos pide más libertad inicial y, por tanto, una T_0 mayor. Con $C(S_0) = 100$:

μ (empeoramiento)	ϕ (probabilidad de aceptarlo)	T_0
0,1 (un 10% peor)	0,5	14,43
1 (el doble de caro)	0,5	144,27
0,1	0,9	94,91
0,1	0,99	994,99

Al multiplicar μ por 10, T_0 se multiplica por 10 (la relación es lineal). En cambio, ϕ actúa a través del logaritmo: cuando ϕ se acerca a 1, $-\ln(\phi)$ se acerca a 0 y T_0 **se dispara**. Además, T_0 es **proporcional al coste de la solución inicial**: así la temperatura queda en la misma escala que los costes del problema. No es lo mismo un empeoramiento de 5 unidades en un problema cuyos costes son del orden de 10 que en uno del orden de 100.000.

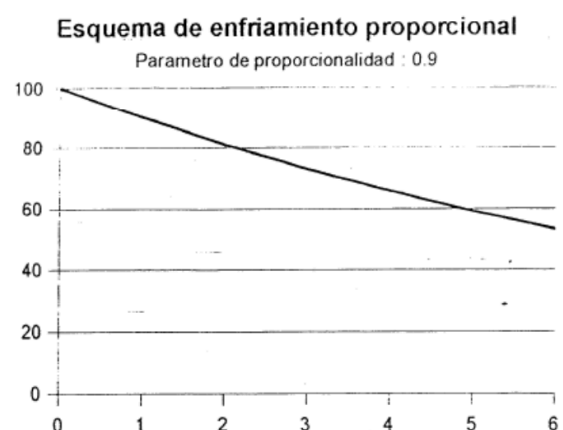
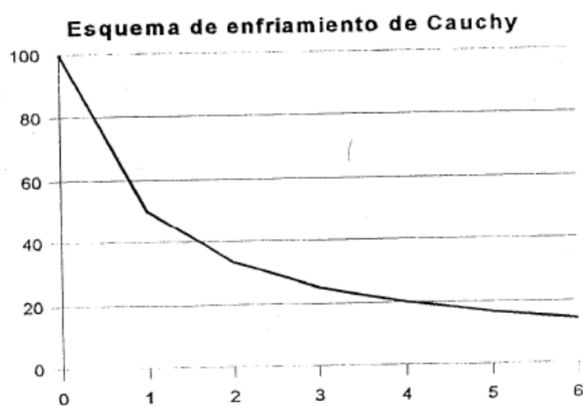
1.5.3 Mecanismo de enfriamiento

El mecanismo de enfriamiento decide **cómo baja la temperatura** entre iteraciones. Hay varias opciones:

Esquema	Fórmula
Temperaturas descendentes fijadas por el usuario	—
Descenso constante	—
Descenso proporcional	$T_{k+1} = \alpha \cdot T_k$, con α cercana a 1 (normalmente [0,8, 0,99])
Criterio de Boltzmann	$T_k = T_0 / (1 + \log(k))$
Esquema de Cauchy	$T_k = T_0 / (1 + k)$
Cauchy modificado	$T_{k+1} = T_k / (1 + \beta \cdot T_k)$

En todos, k es el número de iteración actual. El **Cauchy modificado** sirve para **controlar el número de iteraciones**: si se quiere ejecutar exactamente M iteraciones, basta con tomar

$$\beta = \frac{T_0 - T_f}{M \cdot T_0 \cdot T_f}$$



El esquema de Cauchy (izquierda) baja la temperatura muy rápido al principio y luego despacio; el proporcional con $\alpha = 0,9$ (derecha) la baja de forma mucho más suave.

Para comparar los esquemas, esta tabla muestra la temperatura tras k enfriamientos partiendo de $T_0 = 100$ (con logaritmo neperiano en Boltzmann):

Esquema	$k = 1$	$k = 10$	$k = 100$
Cauchy, $T_0 / (1 + k)$	50	9,09	0,99
Boltzmann, $T_0 / (1 + \ln k)$	100	30,28	17,84
Proporcional, $\alpha = 0,9$	90	34,87	0,0027

Boltzmann enfría lentísimo: tras 100 iteraciones sigue a 17,84, así que explora mucho pero converge muy despacio. **Cauchy** baja muy rápido al principio. El **proporcional** baja de forma suave al principio y termina muy frío.

En el **descenso proporcional**, el valor de α decide la duración de la búsqueda. Para ir de $T_0 = 100$ a $T_f = 1$ hacen falta:

α	Nº de enfriamientos	Comportamiento
0,8	21	Enfriamiento brusco: rápido, pero riesgo de quedarse en un mínimo local ("templar en agua fría")
0,95	90	Término medio
0,99	459	Enfriamiento muy lento: más probabilidad de encontrar el óptimo, pero mucho más tiempo

El **Cauchy modificado** tiene un truco: tomando inversos, la fórmula queda $\frac{1}{T_{k+1}} = \frac{1}{T_k} + \beta$, es decir, $1/T$ **crece una cantidad fija β en cada paso**. Para pasar de $1/T_0$ a $1/T_f$ en exactamente M pasos, basta con repartir la diferencia en M partes iguales, y de ahí sale la fórmula de β . Por ejemplo, con $T_0 = 100$, $T_f = 1$ y $M = 100$: $\beta = \frac{100-1}{100 \cdot 100 \cdot 1} = 0,0099$.

1.5.4 Velocidad de enfriamiento $L(T)$

$L(T)$, el número de vecinos que se generan a cada temperatura, debe ser **lo bastante grande** para que el sistema alcance su **estado estacionario** a esa temperatura. Lo habitual es que sea **fijo**, pero hay una variante que decide mejor cuándo terminar la iteración y enfriar: se enfría en cuanto ocurre una de dos cosas, que se haya **generado** un número máximo de vecinos (**máx_vecinos**) o que se haya **aceptado** un número máximo de vecinos (**máx_éxitos**). Lógicamente, máx_vecinos debe ser mayor que máx_éxitos; una buena proporción es **máx_éxitos = 0,1 · máx_vecinos**.

ANALOGÍA: DEJAR REPOSAR EL METAL

$L(T)$ es el tiempo que el herrero mantiene el metal a cada temperatura antes de bajarla. Si lo baja demasiado pronto, los átomos no llegan a recolocarse y el resultado es peor; si espera demasiado, pierde tiempo sin ganar nada.

Esta variante adapta $L(T)$ sola a cada fase. Con la temperatura **alta** se acepta casi todo, así que se llega enseguida a **máx_éxitos** y se enfría pronto: no merece la pena quedarse mucho tiempo donde todo se acepta. Con la temperatura **baja** casi nada se acepta, así que se agotan los **máx_vecinos** buscando alguna mejora antes de enfriar. Por ejemplo, con máx_vecinos = 100 y máx_éxitos = 10, al principio bastan unos 10 o 12 vecinos por temperatura y al final se prueban los 100.

1.5.5 Condición de parada

En teoría el algoritmo terminaría cuando $T = 0$, pero en la práctica se para cuando T alcanza una **temperatura final** T_f fijada de antemano, tras un **número fijo de iteraciones**, o cuando **no hay**

mejora significativa (o ninguna) durante varias iteraciones. Como es difícil elegir T_f , lo más habitual es fijar el **número de iteraciones**.

Una buena opción es **parar cuando no se ha aceptado ningún vecino** de los $L(T)$ generados en la iteración actual (**num_éxitos = 0**), porque entonces es muy probable que el algoritmo se haya **estancado** y ya no vaya a mejorar. Combinando esta condición de parada con el enfriamiento por **máx_vecinos** y **máx_éxitos** se consigue una búsqueda **equilibrada** que no malgasta recursos.

1.5.6 Resumen: ¿qué pasa si cambio cada parámetro?

Parámetro	Si es demasiado bajo / pequeño	Si es demasiado alto / grande
T_0	Casi no acepta empeoramientos: se comporta como búsqueda local y se atasca en el primer óptimo local	Acepta casi todo: al principio es un paseo aleatorio y gasta iteraciones inútiles
α (proporcional)	Enfría de golpe ($0,8 \rightarrow 21$ pasos de 100 a 1): rápido pero con más riesgo de óptimo local	Enfría muy despacio ($0,99 \rightarrow 459$ pasos): mejores soluciones pero mucho más tiempo
$L(T)$	No da tiempo a explorar cada temperatura: equivale a enfriar demasiado rápido	Cada temperatura se alarga sin mejorar: tiempo perdido
T_f	Sigue iterando cuando ya no acepta nada (tiempo perdido)	Para antes de terminar de afinar la solución
μ, ϕ	T_0 pequeña (ver fila de T_0)	T_0 grande (ver fila de T_0)
Tamaño del entorno	Pasos baratos pero más óptimos locales	Menos óptimos locales pero cada paso es más caro

PARA ENTENDERLO MEJOR

[Algorítmica, Tema 3: Enfriamiento Simulado \(F. Herrera, UGR\)](#) — apuntes con el diseño del SA, esquemas de enfriamiento y ejemplos (PDF; referencia básica del tema). [Recocido simulado \(4 de 8\). Algoritmo Básico I](#) — el algoritmo paso a paso con sus parámetros (vídeo, 7 min; José Manuel Galán).

1.6 Ideas clave

- **Agente:** percibe con sensores y actúa con actuadores. **Racional:** maximiza la medida de rendimiento según percepciones y conocimiento. **Agente = arquitectura + programa.**
- Tipos: **reactivo**, **reactivo basado en modelos**, **basado en objetivos**, **de utilidad**; todos pueden aprender. Resolver un problema: estados, E_0 , acciones, modelo de transiciones, estado objetivo, coste; solución = secuencia $E_0 \rightarrow E_f$, óptima = menor coste.

- **Búsqueda local:** estructura de entornos $N(s)$; **mínimo local** si ningún vecino es mejor. Operadores: **inserción, intercambio, inversión**. La estrategia "el mejor" es **voraz y monótona** y se queda en el primer óptimo local.
- **Metaheurísticas** (Glover, 1986): algoritmos **aproximados de propósito general** para problemas **NP-duros**; no deterministas; trayectoria vs población. Equilibrio **intensificación (explotación) / diversificación (exploración)**.
- **SA:** acepta siempre soluciones mejores y las peores con $P = e^{-\delta/T}$, $\delta = C(s_{vecina}) - C(s_{actual})$. Mayor T o menor $\delta \rightarrow$ más aceptación. Diversifica al principio, intensifica al final. Inspirado en Metropolis (1953) y Kirkpatrick et al. (1983).
- Diseño: representación, T_0 , mecanismo de enfriamiento α , velocidad $L(T)$ y condición de parada.
- $T_0 = \frac{\mu}{-\ln(\phi)} C(S_0)$ (μ en tanto por uno). Enfriamiento proporcional $T_{k+1} = \alpha T_k$ con $\alpha \in [0,8, 0,99]$; Boltzmann, Cauchy y Cauchy modificado ($\beta = \frac{T_0 - T_f}{MT_0 T_f}$ para M iteraciones).
- Enfriar al llegar a máx_vecinos o máx_éxitos ($= 0,1 \cdot \text{máx_vecinos}$); parar cuando $\text{num_éxitos} = 0$.
- Regla práctica: la probabilidad de aceptar un empeoramiento $\delta = T$ es siempre $\approx 0,37$. T_0 baja $\rightarrow$ búsqueda local; T_0 alta $\rightarrow$ paseo aleatorio. α cerca de 1 $\rightarrow$ más lento y mejor.